

Informe de Análisis Dinámico

Samuel Steeven Villagrán
Hernán David Sánchez Alarcón

John David Ibáñez
Docente

Escuela Colombiana De Ingeniería Julio Garavito
Desarrollo Orientado por Objetos
8 de noviembre 2025

1. Resultado inicial:

En esta primera ejecución, la cobertura total alcanzada fue del 50% de instrucciones cubiertas en la clase de pruebas, con fallos detectados en algunos métodos.

Uno de los errores más relevantes ocurrió en el test `accordingSVshouldSortRobotAscnedingByLocation()`

silkroadTotalTest (4 nov 2025 23:00:12)

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 SilkRoad		50%		33%	229	384	689	1,374	96	209	7	22
Total	3,390 of 6,806	50%	233 of 350	33%	229	384	689	1,374	96	209	7	22

silkroadTotalTest (4 nov 2025 23:00:12)

Ilustración 1: Resultados iniciales pruebas de cobertura

silkroadTotalTest

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● <code>accordingSVshouldSortRobotsAscendingByLocation()</code>		23%			n/a	0 1	7 10	0 1		
● <code>accordingSVshouldReturnTotalGainsFromAllRobots()</code>		13%			n/a	0 1	9 10	0 1		
● <code>accordingSVshouldHaveProfitAfterMove()</code>		13%		0%	1 2	7 8	0 1			
● <code>tenderToShop()</code>		25%			n/a	0 1	5 7	0 1		
● <code>accordingSVshouldntAddTwoRobotsInSamePosition()</code>		31%			n/a	0 1	4 5	0 1		
● <code>accordingSVshouldCreateSilkRoadMaratonInput()</code>		77%			n/a	0 1	3 4	0 1		
● <code>shouldCreateSolutions()</code>		93%		50%	1 2	1 4	0 1			
● <code>shouldHaveCorrectAnswers()</code>		100%			n/a	0 1	0 6	0 1		
● <code>shouldCreateSolutionsForEachInput()</code>		100%			n/a	0 1	0 5	0 1		
● <code>accordingSVshouldHave2ShopsAndOneRobot()</code>		100%			n/a	0 1	0 7	0 1		
● <code>shouldGetZeroWhenTheresNoRobots()</code>		100%			n/a	0 1	0 5	0 1		
● <code>accordingSVshouldCheckProfitPermove()</code>		100%			n/a	0 1	0 6	0 1		
● <code>shouldGetTotalTengesiIfTheresNoMovement()</code>		100%			n/a	0 1	0 5	0 1		
● <code>accordingSVshouldSortShopsAscendingByLocation()</code>		100%			n/a	0 1	0 10	0 1		
● <code>shouldGetZeroWhenTheresNoShops()</code>		100%			n/a	0 1	0 5	0 1		
● <code>accordingSVshouldntCreateSilkRoadWithInvalidInput()</code>		100%			n/a	0 1	0 4	0 1		
● <code>accordingSVshouldResetRobotsToInitialPosition()</code>		100%			n/a	0 1	0 7	0 1		
● <code>accordingSVshouldMoveRobot()</code>		100%			n/a	0 1	0 6	0 1		
● <code>accordingSVshouldntAddTwoShopsInSamePosition()</code>		100%			n/a	0 1	0 6	0 1		
● <code>accordingSVshouldCreateSilkRoadEmpty()</code>		100%			n/a	0 1	0 4	0 1		
● <code>accordingSVshouldAddShop()</code>		100%			n/a	0 1	0 4	0 1		
● <code>accordingSVshouldntMoveRobot()</code>		100%			n/a	0 1	0 4	0 1		
● <code>accordingSVshouldAddRobot()</code>		100%			n/a	0 1	0 4	0 1		
● <code>setFighter()</code>		100%			n/a	0 1	0 3	0 1		
● <code>setAutonomous()</code>		100%			n/a	0 1	0 3	0 1		
● <code>accordingSVshouldntCreateSilkRoadWithInvalidLen()</code>		100%			n/a	0 1	0 3	0 1		
● <code>accordingSVshouldGenerateNonEmptySpiralPath()</code>		100%			n/a	0 1	0 3	0 1		
● <code>accordingSVshouldntCreateSilkRoadWithInputHigestThanSilkRoad()</code>		100%			n/a	0 1	0 3	0 1		
● <code>setTender()</code>		100%			n/a	0 1	0 3	0 1		
● <code>setNeverBack()</code>		100%			n/a	0 1	0 3	0 1		
● <code>silkroadTotalTest()</code>		100%			n/a	0 1	0 1	0 1		
Total	150 of 976	84%	3 of 4	25%	2 33	36 158	0 31			

Ilustración 2: Test ejecutados / no ejecutados

1.1 Excepción inesperada:

Observamos en la consola de Eclipse, una excepción arrojada durante la ejecución:

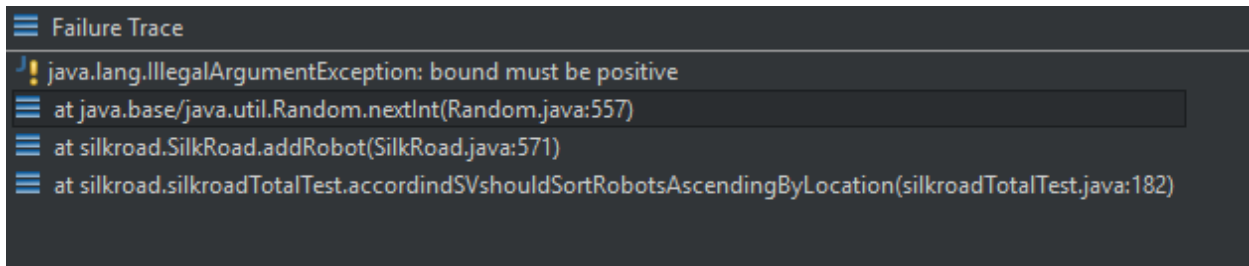


Ilustración 3: Excepción lanzada en las pruebas unitarias

El método `addRobot()` de la clase `SilkRoad` realiza una llamada a `Random.nextInt(bound)`, donde `bound` tenía un valor menor o igual a cero; queriendo acceder a una lista con un índice negativo, lo cual provoca la excepción `IllegalArgumentException`.

```
542  * implemented by this class are known to have short periods in the
543  * sequence of values of their low-order bits. Thus, this special case
544  * greatly increases the length of the sequence of values returned by
545  * successive calls to this method if n is a small power of two.
546  *
547  * @param bound the upper bound (exclusive). Must be positive.
548  * @return the next pseudorandom, uniformly distributed {@code int}
549  *         value between zero (inclusive) and {@code bound} (exclusive)
550  *         from this random number generator's sequence
551  * @throws IllegalArgumentException if bound is not positive
552  * @since 1.2
553  */
554  @Override
555  public int nextInt(int bound) {
556      if (bound <= 0)
557          throw new IllegalArgumentException(BAD_BOUND);
558      int r = next(31);
559      int m = bound - 1;
560      if ((bound & m) == 0) // i.e., bound is a power of 2
561          r = (int)((bound * (long)r) >> 31);
562      else { // reject over-represented candidates
563          for (int u = r;
564              u - (r = u % bound) + m < 0;
565              u = next(31))
566              ;
567      }
568      return r;
569  }
570  /**
```

Ilustración 4: Bloque lanzando excepciones

1.2 Corrigiendo ambiente de pruebas para evitar excepciones:

Se incorporó en cada test la etiqueta `@Before` con el objetivo de asegurar la inicialización correcta de los valores utilizados en las pruebas antes de inicializarlas. Con el fin de no resultar con la lista de Colores vacía.

```

private static final String[] ORIGINAL_COLORS = {
    "red", "black", "blue", "yellow", "magenta", "white",
    "orange", "pink", "cyan", "gray", "darkGray", "brown",
    "maroon", "gold", "darkYellow", "greenTint", "salmon", "darkRed"
};

@Before
public void setUp() {
    SilkRoad.COLORS.clear();
    SilkRoad.COLORS.addAll(Arrays.asList(ORIGINAL_COLORS));
}
    
```

Ilustración 5: Etiqueta agregada para evitar excepciones

1.3 Un segundo resultado inicial:

Una vez implementada la etiqueta volvimos a realizar las pruebas de cobertura y obtuvimos un 67% del total de código en cobertura, además de las pruebas ejecutadas correctamente.

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ SilkRoad	 63,5 %	3.918	2.248	6.166
> src	 63,5 %	3.918	2.248	6.166

Ilustración 6: Resultado cobertura inicia sin excepciones

2. Análisis estado inicial:

Explorando sobre el código, observamos que la clase principal **SilkRoad** tiene un 55.1% de cubrimiento, explorando sobre ella hay una gran cantidad de métodos en los que no hemos testeado su funcionalidad

▼ SilkRoad	 63,5 %	3.918	2.248	6.166
▼ src	 63,5 %	3.918	2.248	6.166
▼ silkroad	 70,5 %	3.628	1.519	5.147
> SilkRoad.java	 55,1 %	1.220	993	2.213
> Tender.java	 0,0 %	0	118	118
> Robot.java	 79,4 %	451	117	568
> Shop.java	 71,2 %	265	107	372
> NeverBack.java	 6,8 %	5	69	74
> SilkRoadContext.java	 85,7 %	349	58	407
> Fighter.java	 23,1 %	6	20	26
> ProgressBar.java	 81,0 %	85	20	105
> SilkRoadC3Tests.java	 98,2 %	432	8	440
> SilkRoadC4Tests.java	 95,3 %	164	8	172
> SilkRoadC2Tests.java	 99,7 %	348	1	349
> Autonomous.java	 100,0 %	6	0	6
> SilkRoadC1Tests.java	 100,0 %	297	0	297
> shapes	 28,6 %	290	725	1.015
> exception	 0,0 %	0	4	4

Ilustración 7: Paquete silkroad cobertura

UNIVERSIDAD

✓ SilkRoad.java		55,1 %	1.220	993	2.213
✓ SilkRoad		55,1 %	1.220	993	2.213
• addShop()		0,0 %	0	106	106
• setNearestRobots()		0,0 %	0	105	105
• resetSilkRoad()		0,0 %	0	94	94
• emptiedStores()		0,0 %	0	88	88
• addRobot()		0,0 %	0	84	84
• moveToMaxGain(boolean)		0,0 %	0	74	74
• makeVisible()		0,0 %	0	54	54
• makeInvisible()		13,0 %	7	47	54
• moveRobotByMeters(int, int, bool)		52,2 %	47	43	90
• removeRobot(int)		0,0 %	0	39	39
• main(String[])		0,0 %	0	37	37
• getRobotWithMajorGain()		0,0 %	0	28	28
• removeShop(int)		0,0 %	0	27	27
• checkAll()		0,0 %	0	20	20
• cleanRoad()		0,0 %	0	18	18
• getSilkRoadInfo()		0,0 %	0	18	18
• resetShops()		0,0 %	0	16	16
• finishSimulator()		0,0 %	0	14	14

Ilustración 8: Métodos no cubiertos en SilkRoad

Además notamos como no solo falta por cubrir los métodos de la clase principal si no también las demás clases del paquete silkroad:

✓ silkroad		70,5 %	3.628	1.519	5.147
> SilkRoad.java		55,1 %	1.220	993	2.213
> Tender.java		0,0 %	0	118	118
> Robot.java		79,4 %	451	117	568
> Shop.java		71,2 %	265	107	372
> NeverBack.java		6,8 %	5	69	74
> SilkRoadContest.java		85,7 %	349	58	407
> Fighter.java		23,1 %	6	20	26
> ProgressBar.java		81,0 %	85	20	105
> SilkRoadC3Tests.java		98,2 %	432	8	440
> SilkRoadCC4test.java		95,3 %	164	8	172
> SilkRoadC2Tests.java		99,7 %	348	1	349
> Autonomous.java		100,0 %	6	0	6
> SilkRoadC1Tests.java		100,0 %	297	0	297

Ilustración 9: clases no cubiertas del paquete silkroad

3. Propuesta inicial:

Proponemos trabajar sobre áreas con 0% de cobertura o en zonas donde esta es demasiado bajas, dado que el porcentaje por cubrir es del 11.5%

Cobertura del 0%:

Trabajar sobre clases con la cobertura del 0%, ya que al ser cortos en código al cubrirlos generará un aumento rápido y fácil que aportará al porcentaje de

Baja cobertura: trabajar sobre con mayor cantidad de instrucciones sin cubrir como SilkRoad con los métodos `addShop()`, `resetSilkRoad()`, `emptied Stores()`, `addRobot()`...

Aplicando Cobertura del 0%:



Aplicando baja cobertura:

Como segunda opción, vamos a cubrir aquellas clases en las que el porcentaje es bajo, a primera vista vemos como SilkRoad llega a 55.1% aunque es más de la mitad sigue siendo bajo para la gran cantidad de métodos que maneja.

Element	Coverage	Covered Instructions
✓ SilkRoad	 77,7 %	5.251
✓ src	 77,7 %	5.251
✓ silkroad	 81,7 %	4.691
> SilkRoad.java	 66,8 %	1.479
> Robot.java	 80,7 %	461
> Shop.java	 81,7 %	304
> SilkRoadContest.java	 85,7 %	349
> Tender.java	 71,9 %	87
> SilkRoadCC4test.java	 96,5 %	496
> Fighter.java	 57,7 %	15
> SilkRoadC3Tests.java	 98,2 %	432
> ProgressBar.java	 94,3 %	99
> SilkRoadC2Tests.java	 99,7 %	348
> Autonomous.java	 100,0 %	6
> NeverBack.java	 100,0 %	74
> SilkRoadC1Tests.java	 100,0 %	541
> shapes	 54,8 %	556
> exception	 100,0 %	4

Ilustración 11: Resultado en baja cobertura

Por los resultados esperados hemos cumplido con nuestro objetivo de cobertura sin necesidad de acudir a testear sobre el paquetes de shapes, ya que el objetivo siempre fue apuntarle a elaborar código elaborado por nosotros y no ajeno, que si se debería hacer pero por fines pedagógicos hemos decidido manejarlo así.